JAVA SHORT NOTES – WITH CODE

Object, Class:

- **Object:**
    - An instance of a class.
    - Represents a real-world entity.
    - Has state and behavior.
- **Class:**
    - Blueprint for objects.
    - Defines attributes (fields) and behaviors (methods) that objects of the class will have.

Class Declaration:

```
class MyClass {
    // Fields (Instance Variables)
    int myField;

    // Methods (Instance Methods)
    void myMethod() {
        // method implementation
    }
}
```

Object as an instance of the class:

```
MyClass obj = new MyClass(); // Creating an instance of MyClass
```

Instance Variables, Instance Methods:

```
class MyClass {
    // Instance Variables
    int x;

    // Instance Method
    void display() {
        System.out.println("Value of x: " + x);
    }
}
```

Overloading Methods:

```
class MathOperations {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }
}
```

Access Modifiers (Public, Private):

- Public: Accessible from any class.
- Private: Accessible only within the class.

Abstraction:

- Abstraction hides complex implementation details, showing only essential features.
- Achieved through abstract classes and interfaces.

Encapsulation:

- Encapsulation bundles data (fields) and methods that operate on the data into a single unit (class).
- Access modifiers control access to encapsulated data.

Constructors:

```
class Person {
    String name;

    // Constructor
    Person(String n) {
        name = n;
    }
}
// Usage:
Person person1 = new Person("John");
```

Access Methods and Variables:

- Accessor methods (getters) and mutator methods (setters) for accessing and modifying private variables.

Object Assignment:

```
MyClass obj1 = new MyClass();
MyClass obj2 = obj1; // obj2 refers to the same object as obj1
```

String Class and Packages:

String Class:

- Immutable sequence of characters.
- Example: String str = "Hello";

Packages:

- Organize classes into namespaces.
- Example: package com.example.myproject;

Implementing Class:

```
interface MyInterface {
    void myMethod();
}


class MyImplementation implements MyInterface {
    public void myMethod() {
        // Implementation of the method
    }
}
```

Static, Not-Static Context:

- Static Context:
    - Belongs to the class, not instance.
    - Example: static int count;
- Not-Static (Instance) Context:
    - Belongs to instances of the class.
    - Example: int x;

Differences: Instance and Class Variables:

- Instance Variables:
    - Belong to instances, unique for each object.
- Class Variables:
    - Shared among all instances of the class.

Casting:

```
double d = 10.5;

int i = (int) d; // Casting double to int
```

Inheritance:

- Inheritance allows a class to inherit properties and behaviors of another class.
- Example: class ChildClass extends ParentClass

Class Extension:

- Extending a class to create a subclass.

Subclasses:

- Classes derived from another class (superclass).

Overriding Methods:

```
class Parent {
    void display() {
        System.out.println("Parent class");
    }
}


class Child extends Parent {
    void display() {
        System.out.println("Child class");
```

```
    }
}
```

Super:

- super refers to the superclass.
- Used to call the superclass's methods or access its fields.

Access Modifiers (Protected):

- Protected:
    - Accessible within the same package or by subclasses.

Polymorphism:

- Polymorphism allows objects of different classes to be treated as objects of a common type.
- Achieved through method overloading and overriding.

Substitution Principle:

- Objects of a superclass can be replaced with objects of a subclass without affecting the correctness of the program.

Dynamic Binding:

- Method binding (selection) happens at runtime.

instanceof:

```
if (obj instanceof MyClass) {
    // obj is an instance of MyClass
}
```

Object Class in Java:

- Object class is the root class for all Java classes.
- All classes inherit from Object by default.

Final Classes and Methods:

- Final Classes:
    - Cannot be extended.
- Final Methods:
    - Cannot be overridden.

Abstract Classes:

- Abstract Classes:
    - Cannot be instantiated.
    - May contain abstract methods.

Interfaces:

- Interfaces:
    - Contract for classes to implement.
    - All methods in interfaces are implicitly abstract.

Collections and Data Structures:

- Array:
    - Fixed-size, ordered collection.
- Set, List, Queue, Map:
    - Interfaces representing different types of collections.
- ArrayList:
    - Dynamic array implementation of List.
- HashSet:
    - Unordered collection of unique elements.
- HashMap:
    - Key-Value pair collection.

Code:

Array:

```
public class ArrayExample {

    public static void main(String[] args) {
```

```
        // Declare and initialize an array
        int[] numbers = {1, 2, 3, 4, 5};

        // Access elements of the array
        System.out.println("Element at index 2: " + numbers[2]);

        // Modify elements of the array
        numbers[2] = 10;

        // Iterate through the array
        for (int i : numbers) {
            System.out.print(i + " ");
        }
    }
}
```

Set, List, Queue, Map (ArrayList, HashSet, HashMap):

```
import java.util.*;

public class CollectionExamples {
    public static void main(String[] args) {
        // ArrayList (List)
        List<String> list = new ArrayList<>();
        list.add("Apple");
        list.add("Banana");
        list.add("Orange");
```

```
        // HashSet (Set)
        Set<Integer> set = new HashSet<>();
        set.add(1);
        set.add(2);
        set.add(3);

        // HashMap (Map)
        Map<String, Integer> map = new HashMap<>();
        map.put("One", 1);
        map.put("Two", 2);
        map.put("Three", 3);

        // Display elements
        System.out.println("List: " + list);
        System.out.println("Set: " + set);
        System.out.println("Map: " + map);

        // Access elements in the Map
        System.out.println("Value for key 'Two': " + map.get("Two"));
    }
}
```

Searching and Sorting in Java using Comparable:

```
class Student implements Comparable<Student> {

    int id;


    public int compareTo(Student s) {

        return this.id - s.id;

    }

}

// Usage:

List<Student> studentList = new ArrayList<>();

Collections.sort(studentList);
```

GUI in Java: AWT and SWING:

- AWT (Abstract Window Toolkit):
    - Basic GUI components:
        - AWT provides a set of components like buttons, text fields, and labels.
        - These components are platform-dependent and use native resources.
- Swing:
    - Extension of AWT with more advanced components:
    - Swing is built on top of AWT and provides a more extensive set of components.
    - Components in Swing are lightweight, offering a consistent look and feel across different platforms.
    - Examples of Swing components include JButton, JtextField, and Jlabel.

Event Handling:

- Delegation Event Handling Model:
    - Components delegate events to event listeners:
        - In the event-driven programming model, components generate events (e.g., button click, mouse movement).
        - Event listeners are objects that handle these events.

- Event Source:
    - Component that generates an event:
        - The event source is the GUI component that triggers an event, such as a button being clicked.
- Event Listener:
    - Object that handles events:
        - Event listeners are objects responsible for responding to specific types of events.
        - For example, a button click event might be handled by a ActionListener.
- Events Handling:
    - Responding to user actions or system events:
        - Events can be triggered by user actions (e.g., button clicks) or system events (e.g., window resizing).
        - Event handling involves writing code to respond appropriately to these events.
        - How Event Handler Works:
            - Code that responds to events:
                - Event handlers contain the logic that executes when a specific event occurs.
                - For example, a button click event handler might perform an action when the button is clicked.

Streams, Predefined Streams:

- Streams:
    - Sequences of data elements:
        - Streams in Java represent sequences of data that can be processed sequentially or in parallel.
        - They are commonly used for I/O operations and working with collections.
- Predefined Streams:
    - System.in, System.out, System.err for input, output, and error streams:
        - System.in: Standard input stream (e.g., keyboard input).
        - System.out: Standard output stream (e.g., console output).
        - System.err: Standard error stream (e.g., error messages).

Exception:

- An unexpected event during program execution:
    - Exceptions represent unexpected situations or errors during the execution of a program.
    - They can be handled using try-catch blocks.

Try and Catch Statements:

```
try {
    // code that may throw an exception
} catch (ExceptionType e) {
    // handle the exception
}
```

Multiple Catch Statements:

```
try {
    // code that may throw different exceptions
} catch (ExceptionType1 e) {
    // handle ExceptionType1
} catch (ExceptionType2 e) {
    // handle ExceptionType2
}
```

Using Finally:

```
try {
    // code that may throw an exception
} catch (ExceptionType e) {
    // handle the exception
} finally {
    // code that will always be executed
```

```
}
```

Declaring Exceptions:

```
void myMethod() throws MyException {
    // method implementation
}
```

Throwable Class:

- Throwable:
    - Base class for all exceptions and errors.

Checked and Unchecked Exceptions:

- Checked Exceptions:
    - Checked at compile-time. Extends Exception but not RuntimeException.
- Unchecked Exceptions:
    - Extends RuntimeException.
    - Not checked at compile-time.

Throwing Exceptions:

```
void myMethod() {
    if (errorCondition) {
        throw new MyException("Error message");
    }
}
```

Custom Exceptions:

```
class MyException extends Exception {
    MyException(String message) {
        super(message);
    }
}
```

Threads, Multithreading:

Implementing Threads:

```
class MyThread extends Thread {

    public void run() {

        // code to be executed in the new thread

    }

}
```

- 2 Threading Mechanisms (Thread and Runnable):
    - Extending Thread class or implementing Runnable interface.
- Create Multiple Threads:
    - Instantiate and start multiple threads.
- Life Cycle of a Thread:
    - New, Runnable, Blocked, Waiting, Timed Waiting, Terminated.
- Thread States:
    - Represent different stages in a thread's life cycle.
- Terminating Threads:
    - stop() method or reaching the end of run() method.
- yield(), sleep():
    - yield(): Gives up the CPU to let other threads run.
    - sleep(): Pauses the thread for a specified period.
- Concurrent Access to Data:
    - Synchronization to avoid race conditions.
- Synchronization:
    - Ensures only one thread accesses shared data at a time.
- wait(), notify(), notifyAll():
    - Used for inter-thread communication.
- Suspending and Resuming Threads:
    - Deprecated methods like suspend() and resume(). Considered unsafe.